

ΕΡΓΑΣΙΑ : ΑΝΑΓΝΩΡΙΣΗ ΠΡΟΤΥΠΩΝ ΚΑΙ ΜΗΧΑΝΙΚΗ ΜΑΘΗΣΗ

Μιχαήλ Μπαλτζάκης : 10989

Άγγελος Γούργαρης : 11164

ΜΕΡΟΣ Α

Σε αυτό το μέρος της εργασίας θα ασχοληθούμε με τη μέθοδο της μέγιστης πιθανοφάνειας

Στη διάθεσή μας έχουμε ένα σύνολο δεδομένων 300 sample δύο διαστάσεων

Έχουμε 3 διαφορετικές κλάσεις. Κάθε κλάση αφορούν 100 samples από τα δεδομένα μας

Υποθέτουμε ότι τα δεδομένα κάθε κλάσης προέρχονται από κανονική κατανομή

Στόχος μας είναι να χρησιμοποιήσουμε τη μέθοδο της μέγιστης πιθανοφάνειας ώστε να εκτιμήσουμε τις παραμέτρους κάθε κανονικής κατανομής και στη συνέχεια να τις σχεδιάσουμε σε ένα ενιαίο 3D plot

Παράμετροι κανονικής κατανομής

Σε μια πολυμεταβλητή κανονική κατανομή έχουμε δύο παραμέτρους:

Το διάνυσμα μ που περιλαμβάνει το μέσο όρο κάθε διάστασης και έχει μέγεθος d όταν είμαστε στις d διαστάσεις

Τον πίνακα συνδιακύμανσης Σ που έχει μέγεθος $d \times d$ όταν βρισκόμαστε στις d διαστάσεις. Τα στοιχεία στην κύρια διαγώνιο αντιπροσωπεύουν τη διακύμανση (variance) της κάθε μεταβλητής ξεχωριστά. Τα υπόλοιπα στοιχεία αντιπροσωπεύουν τη συνδιακύμανση (covariance) μεταξύ ζευγών μεταβλητών.

Στο πρόβλημά μας βρισκόμαστε στις 2 διαστάσεις

Εκτίμηση παραμέτρων με MLE

Παρακάτω θα παραθέσουμε τους τύπους εκτίμησης των παραμέτρων όταν γνωρίζουμε ότι τα δεδομένα μας προέρχονται από κανονική κατανομή:

$$\hat{\mu}_{MLE} = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\hat{\Sigma}_{MLE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})(x_i - \hat{\mu})^T$$

Όπου N (100 στη περίπτωση μας) ο αριθμός των δειγμάτων για τη συγκεκριμένη κλάση της οποίας τις παραμέτρους προσπαθούμε να εκτιμήσουμε. Το ίδιο και το μ . Πρέπει να επισημάνουμε ότι είναι υποχρεωτικός ο υπολογισμός του μ πριν προβούμε στον υπολογισμό του Σ .

Αποτελέσματα

Εφαρμόζοντας τους τύπους της προηγούμενης διαφάνειας στα δεδομένα μας πήραμε τα εξής αποτελέσματα:

Στον κώδικα για τον υπολογισμό του Σ εφαρμόσαμε τον τύπο :

$$(X_centered^T @ X_centered)/N$$

Όπου ο X είναι πίνακας σε μορφή $N \times d$ και ο $X_centered$ είναι ο πίνακας X έχοντας αφαιρέσει με broadcast από κάθε γραμμή το διάνυσμα των μέσων όρων. Το αποτέλεσμα είναι ο Σ διάστασης $d \times d$

For Class 0:

$$\mu = [29.20903802 \quad 16.84988855]$$

$$\Sigma = \begin{bmatrix} 46.98591258 & 22.90532295 \\ 22.90532295 & 48.63133559 \end{bmatrix}$$

For Class 1:

$$\mu = [40.19766916 \quad 34.28109099]$$

$$\Sigma = \begin{bmatrix} 9.42390491 & 11.48971771 \\ 11.48971771 & 20.10805596 \end{bmatrix}$$

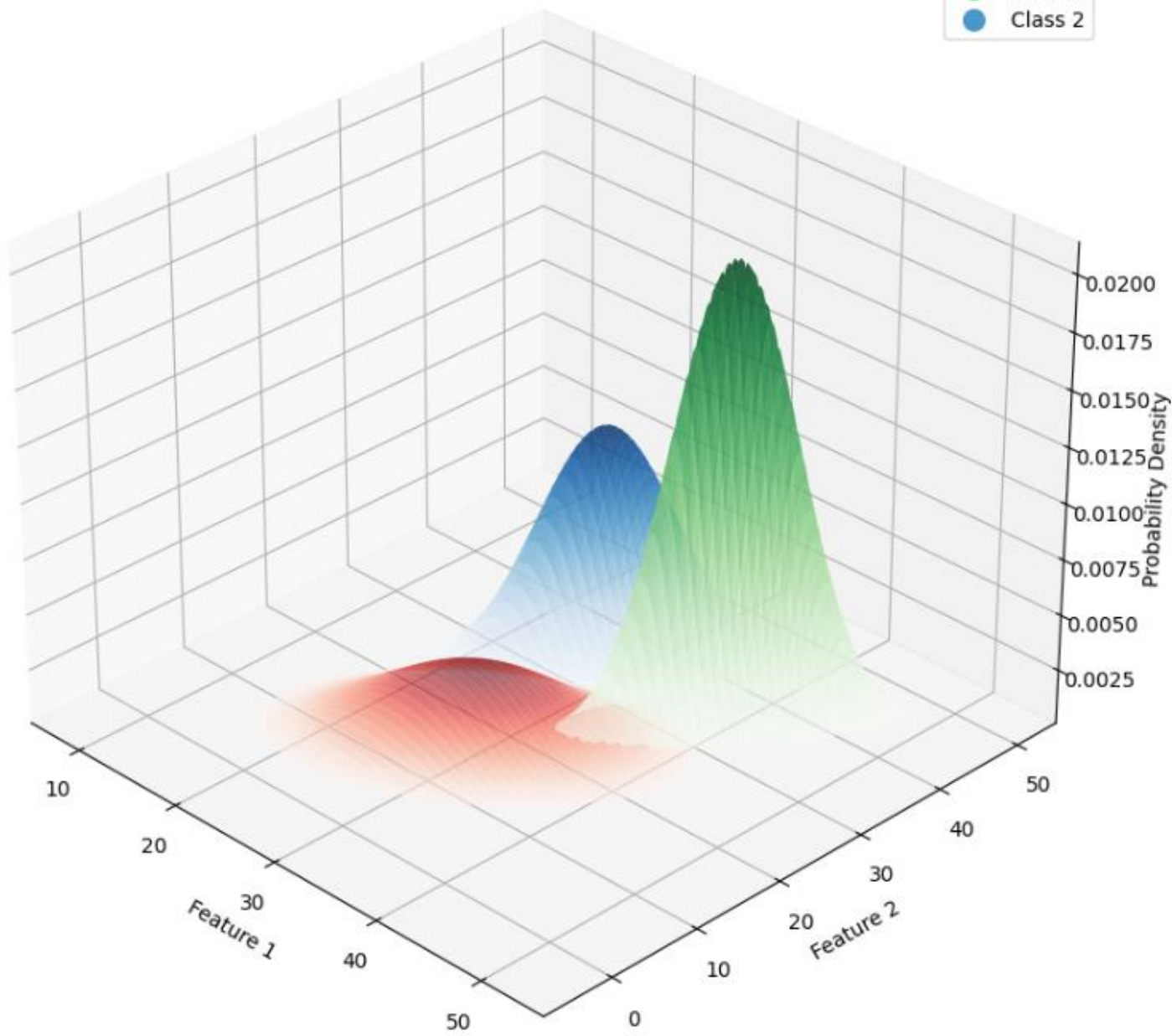
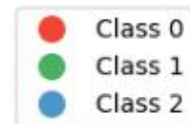
For Class 2:

$$\mu = [27.55232503 \quad 34.79318037]$$

$$\Sigma = \begin{bmatrix} 13.96874391 & 11.77433636 \\ 11.77433636 & 25.28848192 \end{bmatrix}$$

Το @ είναι το σύμβολο του πολ/μου πινάκων στην python. Παρακάτω βλέπουμε το διάγραμμα των κατανομών

3D Multivariate Gaussian Distributions



Παρατηρούμε ότι η κλάση 0 είναι πολύ απλωμένη ενώ οι άλλες δύο κλάσεις είναι πιο μαζεμένες. Αυτό οφείλεται στις μεγάλες τιμές του πίνακα συνδιακύμανσης για τη κλάση 0 σε συνδυασμό με τις μικρότερες τιμές για τις κλάσεις 1 και 2. Η θέση της κορυφής της κάθε κατανομής προκύπτει από το διάνυσμα των μέσων όρων μ όπου στο σημείο που προσδιορίζει το διάνυσμα έχουμε το μέγιστο.

ΜΕΡΟΣ Β

Σε αυτό το μέρος θα ασχοληθούμε με τη μέθοδο parzen windows χρησιμοποιώντας δύο διαφορετικά kernels:

Gaussian kernel

Hypercube kernel

Στη διάθεσή μας έχουμε 200 μονοδιάστατα samples

Μας ζητείται χρησιμοποιώντας τη μέθοδο Parzen να εκτιμήσουμε τη συνάρτηση πυκνότητας πιθανότητας της κατανομής των δεδομένων.

Έπειτα, αν υποθέσουμε ότι τα δεδομένα μας προέρχονται από τη μονοδιάστατη κανονική κατανομή $N(1, 4)$ μας ζητείται να βρούμε το καλύτερο h (υπερπαράμετρος) και να δημιουργήσουμε ένα διάγραμμα που να δείχνει τη σχέση h και σφάλματος μεταξύ πραγματικής και προβλεπόμενης πιθανοφάνειας.

Μέθοδος Parzen με hypercube kernel:

Στη μέθοδο αυτή για να εκτιμήσουμε το $p(x)$ σε ένα τυχαίο σημείο x ουσιαστικά μετράμε πόσα samples θα πέσουν μέσα σε ένα υπερκύβο που έχει πλευρά h και κέντρο το x . Έπειτα διαιρούμε με τον αριθμό των δειγμάτων και στη συνέχεια διαιρούμε με το h υψωμένο στον αριθμό των διαστάσεων που δουλεύουμε. Η $p(x)$ έχει ανώμαλη μορφή με σκαλοπάτια.

$$\hat{p}(x) \approx \frac{1}{h^d} \left(\frac{1}{N} \sum_{i=1}^N \varphi \left(\frac{x_i - \bar{x}}{h} \right) \right) \quad \varphi \left(\frac{x_i - \bar{x}}{h} \right) = \begin{cases} 1, & \forall j, |x_{ij} - \bar{x}_j| \leq \frac{h}{2} \\ 0, & otherwise \end{cases}$$

Μέθοδος Parzen με gaussian kernel:

Εδώ τα πράγματα αλλάζουν και για να εκτιμήσουμε το $p(x)$ παίρνουμε το μέσο όρο από N gaussian που η καθεμία έχει κέντρο διαφορετικό σημείο του training set που έχουμε στη διάθεσή μας. Το h παίζει το ρόλο του σ . Η $p(x)$ έχει ομαλή μορφή.

$$\hat{p}(x) = \frac{1}{N} \sum_{i=1}^n \frac{1}{(2\pi)^{\frac{d}{2}} h^d} \exp \left(-\frac{(x - x_i)^T (x - x_i)}{2h^2} \right)$$

Στο πρόβλημά μας επειδή είμαστε στη μία διάσταση έχουμε $d = 1$

Εκτίμηση συνάρτησης πυκνότητας πιθανότητας της κατανομής των δεδομένων

Εδώ έχουμε υπολογίσει και τις δύο συναρτήσεις και έχουμε επιλέξει να αναπαραστήσουμε σε τύπο τη συνάρτηση του gaussian kernel. Η συνάρτηση δίνεται από τον τύπο:

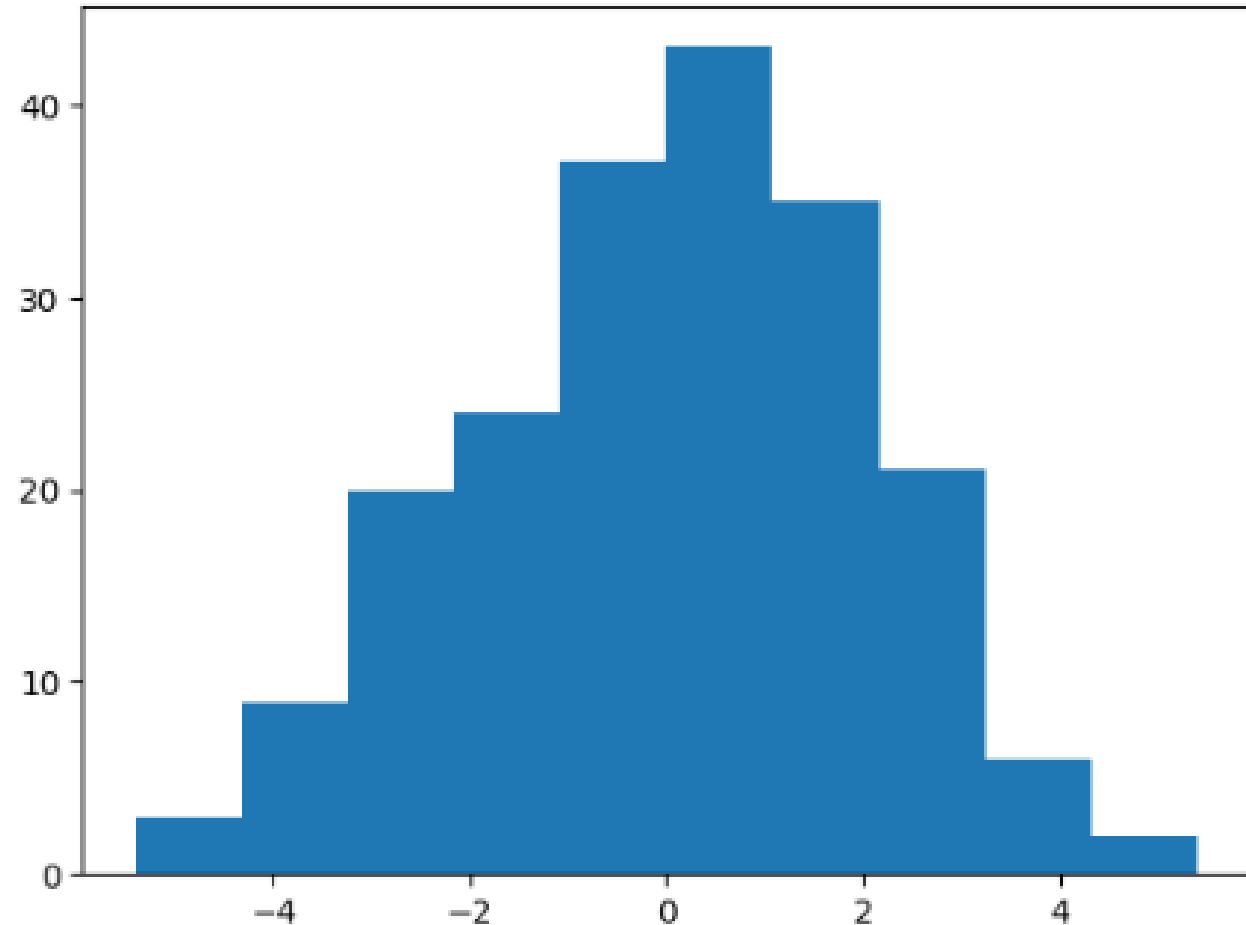
$$0.005 \sum_{i=1}^{200} \frac{\sqrt{2} e^{-\frac{(x-x_{i,j})^2}{2h^2}}}{2\sqrt{\pi}h}$$

Ενώ η πραγματική πιθανοφάνεια αν υποθέσουμε ότι τα δεδομένα μας προέρχονται από την $N(1, 4)$ δίνεται από τον τύπο:

$$\frac{1}{2\sqrt{2\pi}} e^{-\frac{(x-1)^2}{8}}$$

Ιστόγραμμα δεδομένων

Εδώ βλέπουμε ότι τα δεδομένα μας φαίνεται όντως να προκύπτουν από κανονική κατανομή με μέσο όρο 1 και τυπική απόκλιση 2. Έχουμε λίγα δεδομένα για αυτό το ιστόγραμμα έχει αυτή την ‘οδοντωτή’ και λίγο unbalanced μορφή



Εύρεση βέλτιστου h

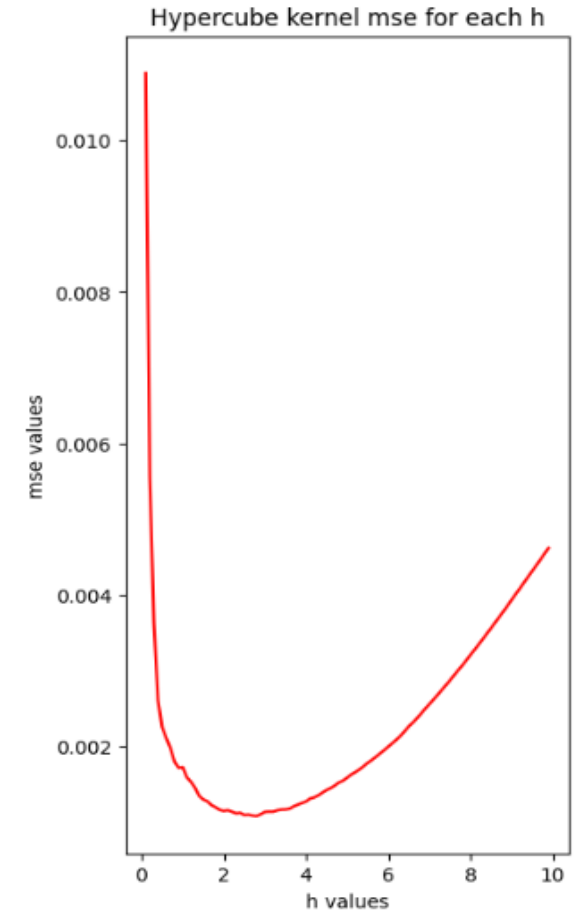
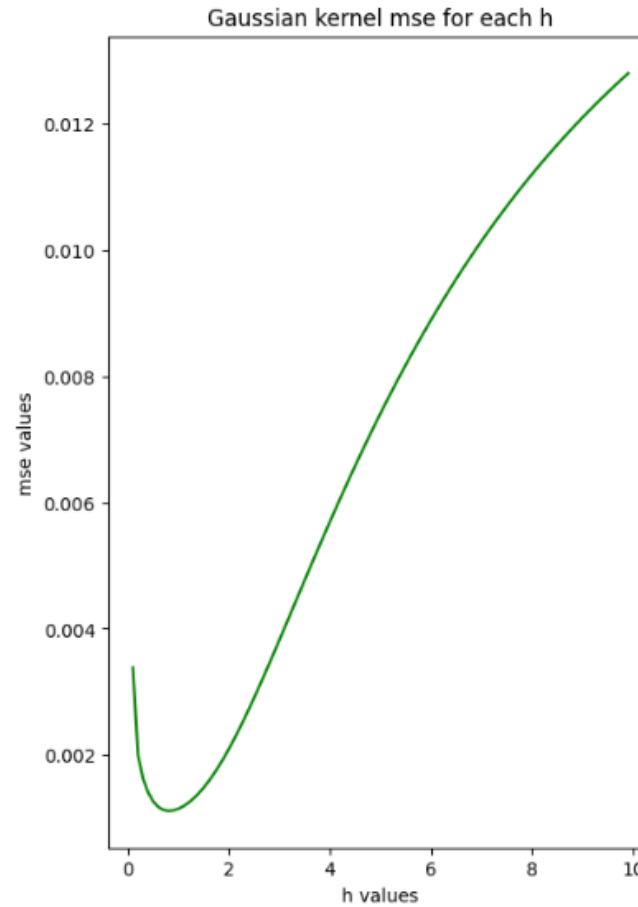
Για την εύρεση του βέλτιστου h τρέξαμε τον Parzen στο $[0.1, 10]$ με βήμα 0.1 και κρατήσαμε το h το οποίο μας ελαχιστοποιούσε το μέσο τετραγωνικό σφάλμα της πραγματικής με την προβλεπόμενη πιθανοφάνεια. Επαναλάβαμε τη διαδικασία για κάθε kernel ξεχωριστά. Παρακάτω βλέπουμε τα αποτελέσματα:

```
lowest_mse = 0.0011 while best_h = 0.80 for gaussian kernel  
lowest_mse = 0.0011 while best_h = 2.80 for hypercube kernel
```

Παρατηρούμε πως και τα δύο kernels έρχονται το ίδιο κοντά στη προσέγγιση της πραγματικής πιθανοφάνειας με διαφορετικά h , η gaussian με μικρότερο. Αυτό συμβαίνει επειδή είναι σαν καμπάνα απλώνεται ομαλά και έχει ουρές έτσι μπορεί να καλύψει τα κενά ανάμεσα στα σημεία χωρίς να χρειάζεται τεράστιο πλάτος. Από την άλλη ο hypercube χρησιμοποιεί μεγαλύτερο h επειδή ο κύβος είναι απότομος και χρειάζεται να είναι αρκετά φαρδύς ώστε να συμπεριλάβει αρκετά σημεία και να λειάνει τα σκαλοπάτια ώστε να μοιάσει στην καμπύλη.

Διαγράμματα σχέσης h με το mse

Παρατηρούμε ότι και οι 2 καμπύλες έχουν την ίδια μορφή. Είναι κυρτές με μοναδικό ολικό ελάχιστο στο σημείο όπου έχουμε το καλύτερο h για το πρόβλημά μας. Αυτό συμβαίνει καθώς δε θέλουμε να είναι ούτε πολύ μεγάλο ούτε πολύ μικρό το h ώστε να μη χάνουμε ακρίβεια στη καμπύλη μας αλλά ούτε και να έχουμε πολύ μεγάλα 'spikes' που εκτοξεύουν τη τιμή της συνάρτησης. Το $h = 0.8$ και $h = 2.8$ είναι η 'χρυσή τομή' σε κάθε περίπτωση.



ΜΕΡΟΣ Γ

Στο μέρος αυτό θα υλοποιήσουμε ένα ταξινομητή KNN

Θα χρησιμοποιήσουμε ένα training set που αποτελείται από 50 samples με 2 features ανά sample

Στόχος μας είναι η ταξινόμηση σε 2 κλάσεις

Έλεγχος απόδοσης του μοντέλου με ένα test set των 50 samples

Δομή υλοποίησης

Έχουμε 3 κύριες συναρτήσεις:

`eucl(x, trainData)` που επιστρέφει την ευκλείδια απόσταση του `x` από όλα τα σημεία του `trainData`.

`neighbors(x, trainData, k)` όπου `k` είναι ο αριθμός των γειτόνων που θέλουμε να ελέγχει το μοντέλο μας πριν πάρει απόφαση. Η `neighbors` χρησιμοποιεί την `eucl` ώστε να υπολογίσει την απόσταση του `x` από κάθε σημείο του `trainData`, ακόμη ταξινομεί τις αποστάσεις σε φθίνουσα σειρά και επιστρέφει τα `k` κορυφαία σημεία του `trainData` δηλαδή τους `k` κοντινότερους γείτονες του `x` ανεξαρτήτου κλάσης. Επιστρέφει τα `indices` των γειτόνων στο `set` και όχι τις αποστάσεις.

`predict(x, trainData, k, extra)` χρησιμοποιεί τη `neighbors` και επιστρέφει δυο πιθανότητες, τη πιθανότητα κάποιο `sample` να ανήκει στη κλάση 0 και τη πιθανότητα κάποιο `sample` να ανήκει στη κλάση 1. Ακόμη διευθετεί τη περίπτωση ισοπαλίας.

Τέλος σχεδιάζουμε τα όρια απόφασης του καλύτερου ταξινομητή(εκείνου με το καλύτερο `k`)

Έχουμε υλοποιήσει **2 παραλλαγές** του KNN, τον **voting KNN** και τον **density estimator KNN** οπότε στον κώδικα όπου βλέπουμε `extra` αναφερόμαστε στον `density estimator KNN`

Λειτουργία KNN

Ο KNN voting απλώς αποθηκεύει το πλέγμα των training δεδομένων, δεν κάνει κανένα υπολογισμό πάνω σε αυτά. Όλη η δουλειά γίνεται όταν έρθει ένα καινούριο Test sample για ταξινόμηση. Εκεί ο KNN υπολογίζει τους k κοντινότερους γείτονες του σημείου ανεξαρτήτου κλάσης και έπειτα βλέπει πόσοι ανήκουν στη μία κλάση πόσοι στην άλλη και ταξινομεί σε εκείνη με τους περισσότερους.

Ο KNN density estimator και αυτός απλώς αποθηκεύει το πλέγμα των training δεδομένων, δεν κάνει κανένα υπολογισμό πάνω σε αυτά. Όταν έρθει ένα καινούριο Test sample για ταξινόμηση βρίσκει τους k κοντινότερους γείτονες κάθε κλάσης και ταξινομεί σύμφωνα με τον παρακάτω τύπο:

$$\frac{V_2}{V_1} > \frac{N_1}{N_2} \frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}} \frac{P(\omega_2)}{P(\omega_1)}$$

Όπου το V είναι ο όγκος της υπερσφαίρας(αφού δουλεύουμε με ευκλείδειες αποστάσεις) της οποίας η ακτίνα καθορίζεται από την ευκλείδεια απόσταση του πιο μακρινού γείτονα από το test sample. N1 και N2 είναι ο αριθμός των δειγμάτων κάθε κλάσης στο training set, τα λ προκύπτουν από τον πίνακα κόστους στη περίπτωση που έχουμε βάλει διαφορετικά κόστη στα λάθη μας και τα P(ω1) και P(ω2) είναι οι εκ των προτέρων πιθανότητες να συναντήσουμε κάποια κλάση. Ταξινομεί σε κλάση 1 αν το V2/V1 είναι μεγαλύτερο από το threshold του δεξιού μέλους.

Γενικά έχουμε ακολουθήσει Vectorized λύση ώστε να είμαστε όσο πιο γρήγοροι γίνεται.

Στα διαγράμματα της επόμενης διαφάνειας φαίνεται η σχέση ακρίβειας k για τα δύο μοντέλα μας.

Παρατηρούμε ότι και τα 2 δύο μοντέλα ακολουθούν μια τεθλασμένη πορεία. Ειδικά στον KNN voting έχει βοηθήσει πολύ σε αυτό ο τρόπος που επιλύουμε τις ισοπαλίες γιατί αρχικά όταν δίναμε το 50-50 τυχαία σε μία από τις 2 κλάσεις εισήγαμε bias και στα ζυγά k είχαμε πρόβλημα. Όταν δίναμε το 50-50 στη κλάση 0 μάλιστα είχαμε μια βελτίωση της τάξεως του 2% αλλά δε κρατήσαμε τη λύση καθώς η λύση ήταν βέλτιστη μόνο για το συγκεκριμένο set λόγω του bias που βοηθούσε. Επίλυση στις ισοπαλίες κάνουμε μετρώντας τη συνολική απόσταση των σημείων της κλάσης 0 και της κλάσης 1 από το test sample που θέλουμε να ταξινομήσουμε. Ταξινομούμε στην κλάση της οποίας η συνολική απόσταση είναι η μικρότερη.

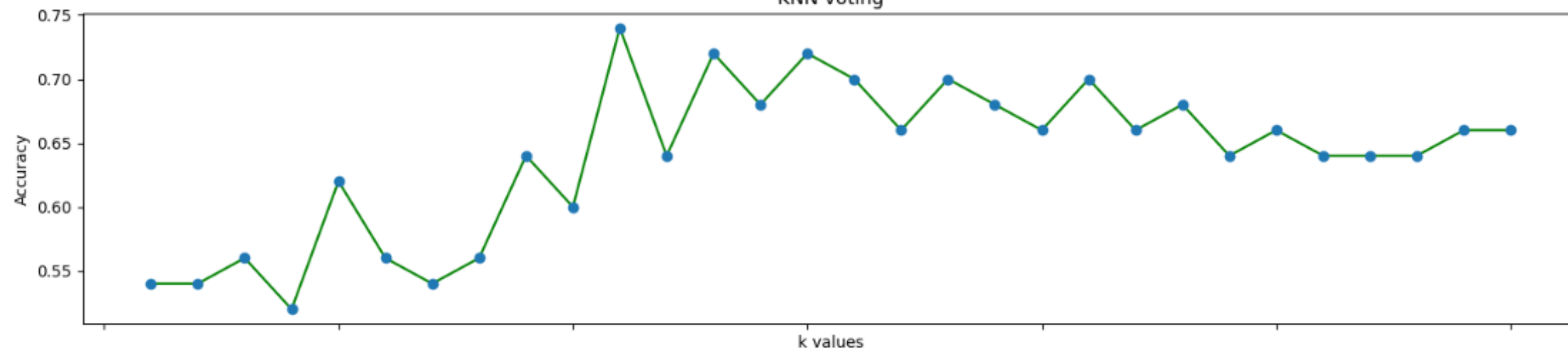
Στον density estimator KNN δεν έχουμε αυτό το πρόβλημα καθώς είναι μηδενική η πιθανότητα ο μακρινότερος γείτονας που αντιστοιχεί σε κλάση 0 να απέχει ίδια ακριβώς απόσταση, σε ακρίβεια δεκαδικού ψηφίου, με τον μακρινότερο γείτονα που αντιστοιχεί σε κλάση 1 από το Test sample.

Πιάνουμε ίδιο ποσοστό ακρίβειας 74% για $K=6$ στον voting και για $K=11$ στον density estimator

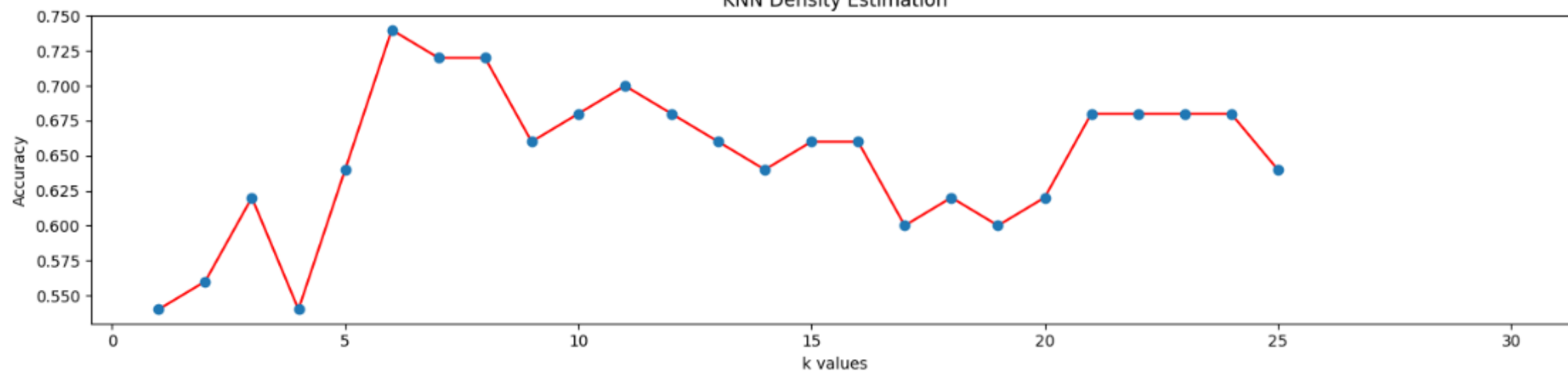
max_accuracy = 0.74 for k = 11

max_accuracy_extra = 0.74 for k = 6

KNN voting



KNN Density Estimation



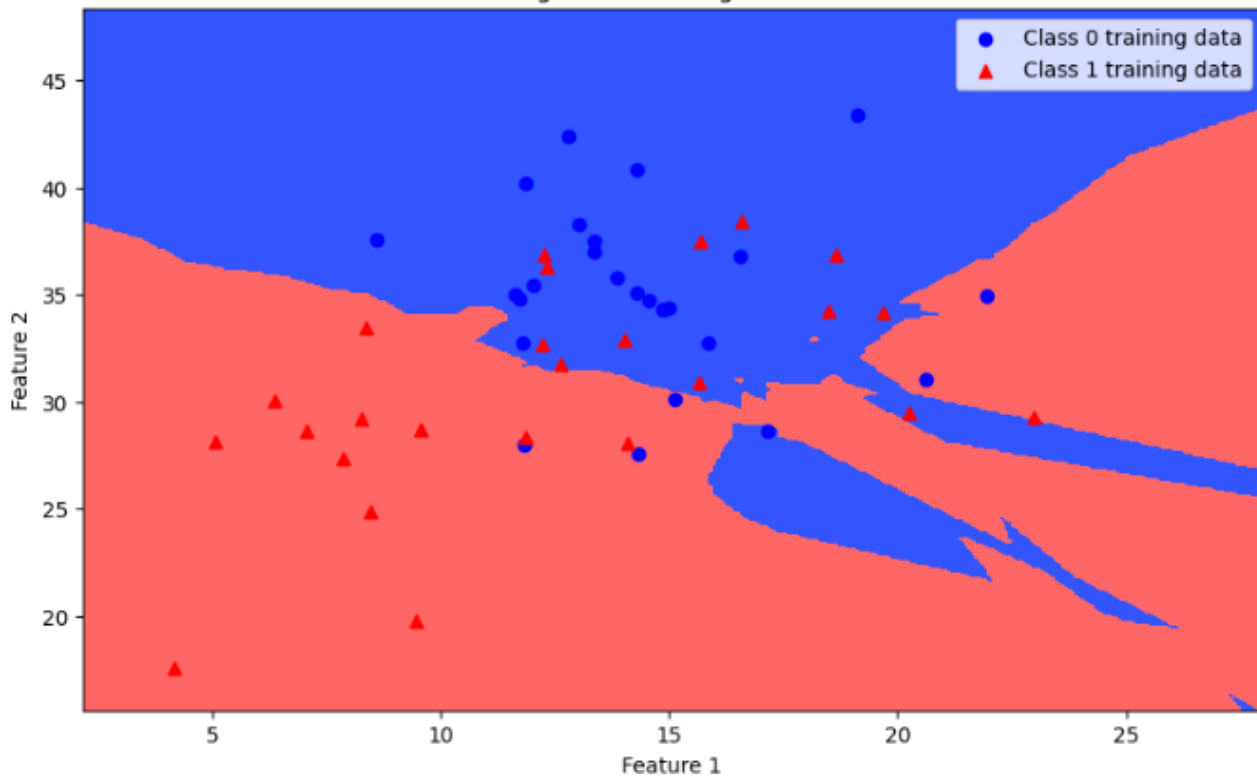
Περιοχές απόφασης

Τέλος έχουμε τη σχεδίαση των περιοχών απόφασης οι οποίες για το καλύτερο K κάθε μοντέλου βγαίνουν οι ίδιες. Θα δούμε όμως ότι διαφέρουν όταν διαλέγουμε ίδιο αριθμό γειτόνων ($k_{\text{voting}} = 2k_{\text{density}}$)

Έχουμε επιλέξει grid 90.000 στοιχείων ώστε να έχουμε καλή ανάλυση αλλά και ταχύτητα στον σχεδιασμό του διαγράμματος. Εδώ είναι χρήσιμη και η vectorized λύση καθώς μας υπολογίζει γρήγορα το Label του καθένα από τα 90.000 σημεία.

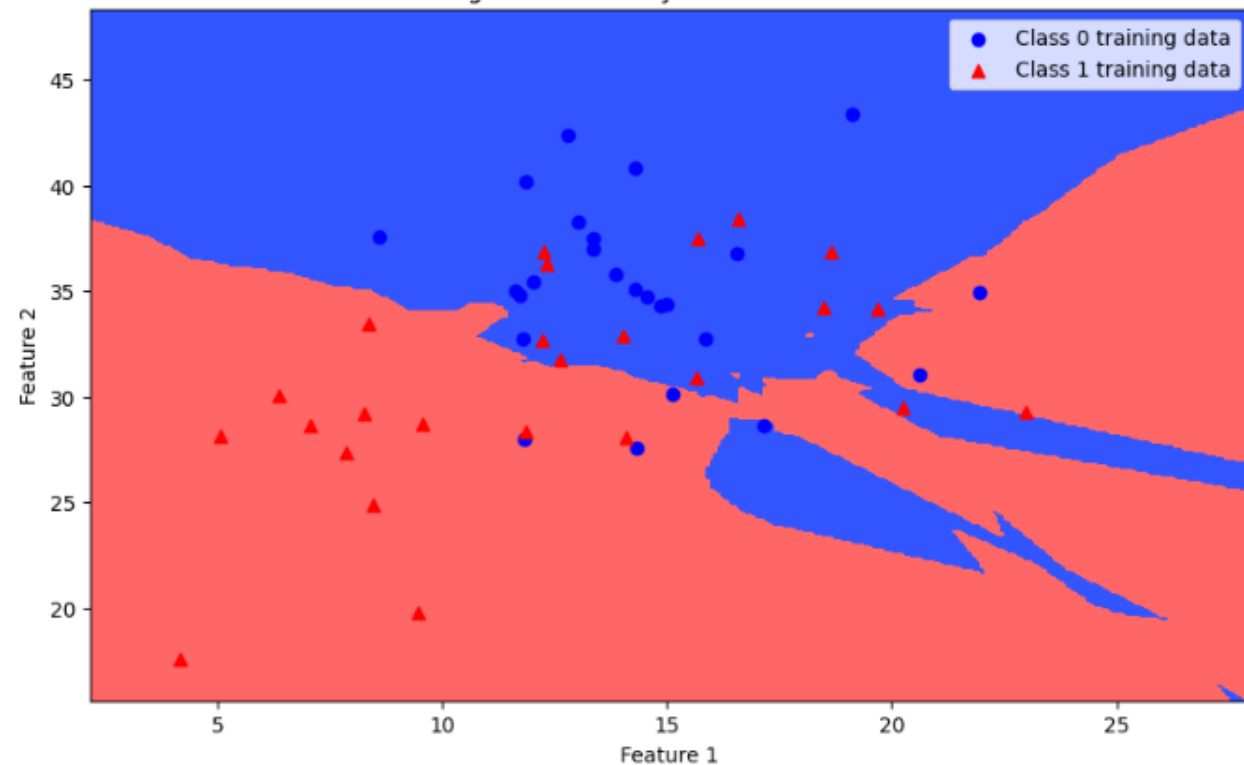
Στο διάγραμμα στην επόμενη διαφάνεια παρατηρούμε ότι έχουμε χωριστά νησάκια και χερσονήσους. Ο KNN μπορεί να λύσει non-linear classification προβλήματα.

Decision regions for voting KNN with $k = 11$

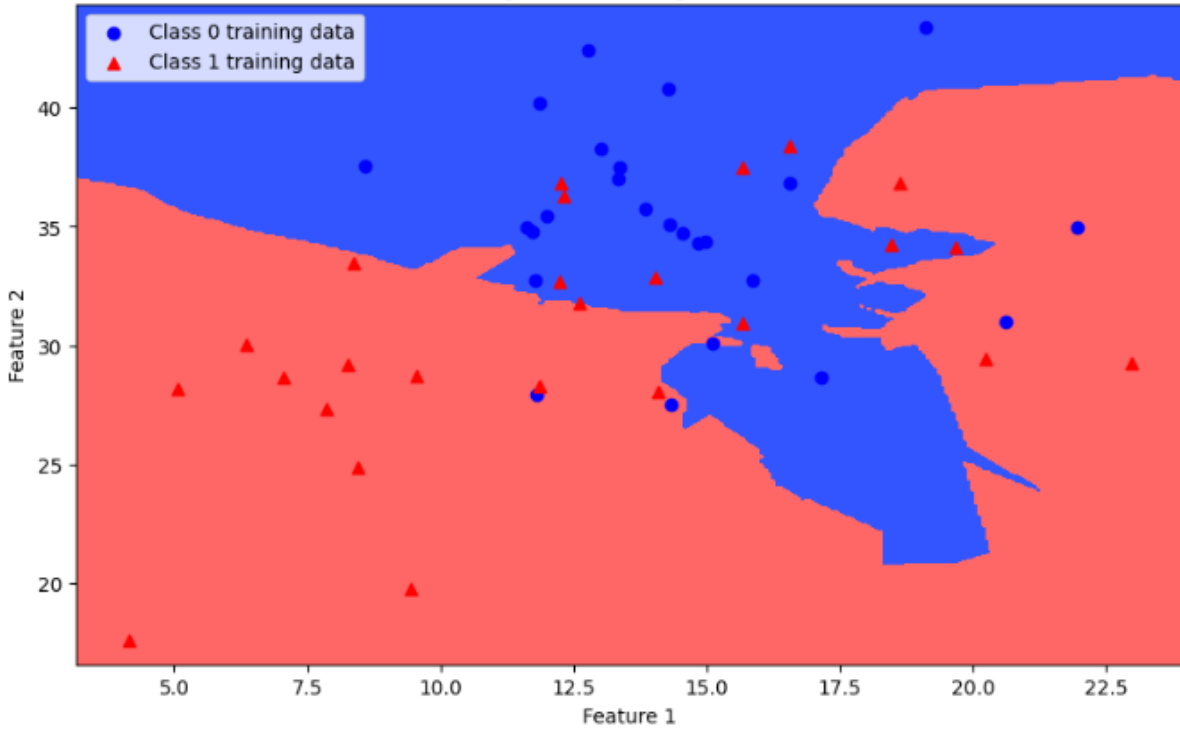


Εδώ βλέπουμε ίδιες περιοχές απόφασης για
τα καλύτερα k

Decision regions for density estimation KNN with $k = 6$

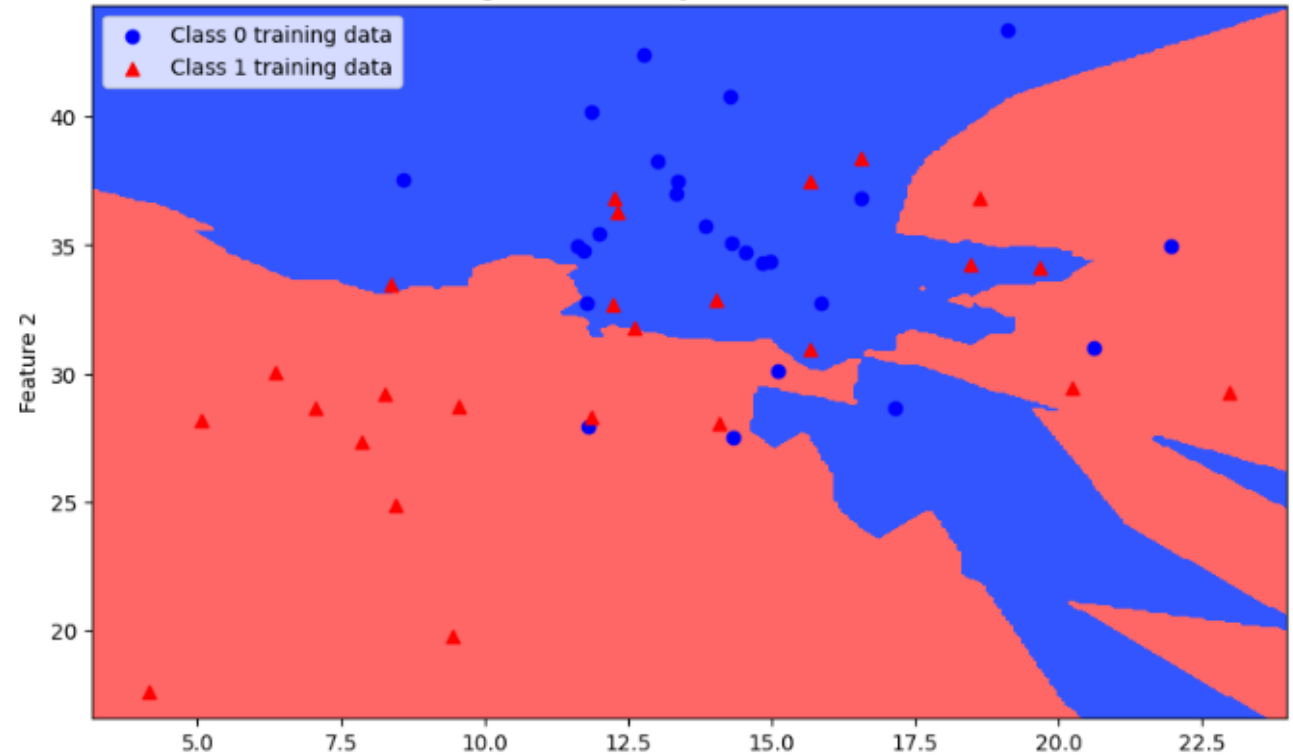


Decision regions for voting KNN with $k = 10$



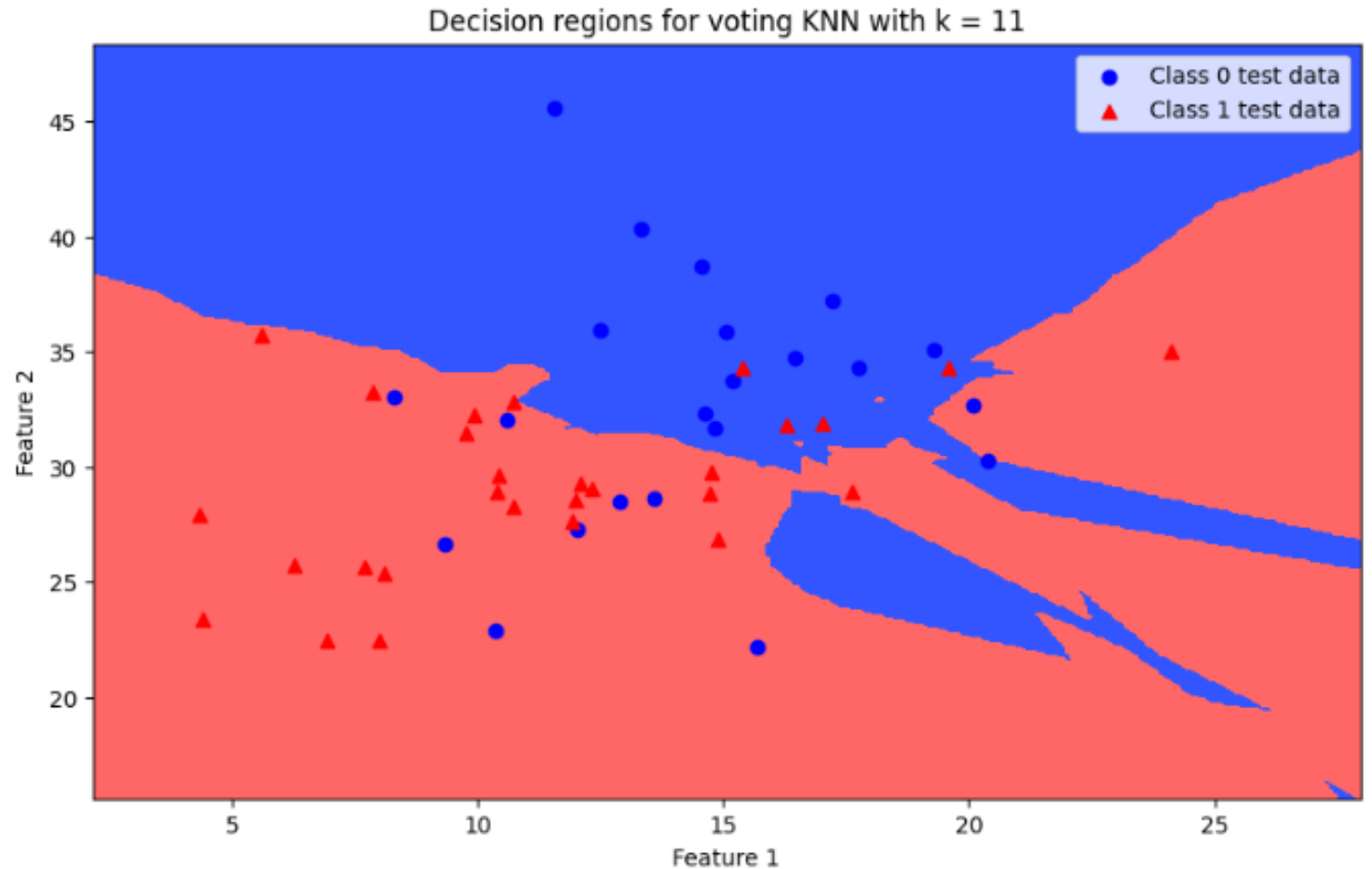
Εδώ βλέπουμε διαφορετικές περιοχές απόφασης για $k_{\text{voting}} = 2k_{\text{density}}$

Decision regions for density estimation KNN with $k = 5$



Οπτικοποίηση λαθών

Αυτή τη φορά στο διάγραμμα με μπλε χρώμα είναι τα test sample που ανήκουν στη κλάση 0 ενώ με κόκκινο είναι τα test sample που ανήκουν στην 1. Μπορούμε να δούμε τα λάθη που έχει κάνει το μοντέλο μας αν εστιάσουμε στα μπλε σημεία που ανήκουν στη κόκκινη περιοχή και στα κόκκινα σημεία που ανήκουν στη μπλε περιοχή.



ΜΕΡΟΣ Δ

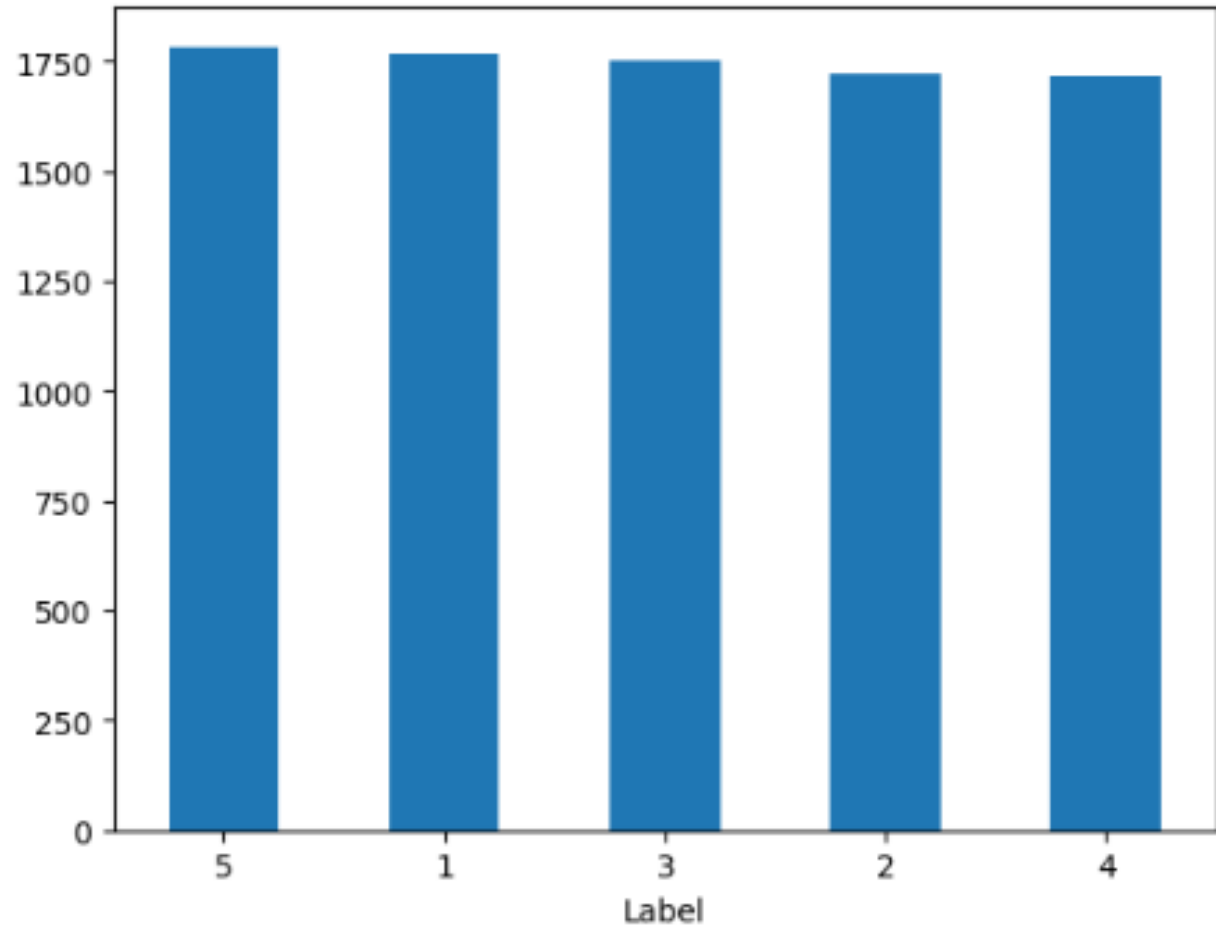
Σε αυτό το μέρος της εργασίας έχουμε ένα πρόβλημα classification σε 5 κλάσεις .

Έχουμε στη διάθεσή μας ένα training set το οποίο αποτελείται από 8743 δείγματα και 224 χαρακτηριστικά ανά δείγμα.

Στόχος είναι η εύρεση ενός αλγορίθμου ταξινόμησης που να χωρίζει με όσο γίνεται μεγαλύτερη ορθότητα τις 5 κλάσεις μας.

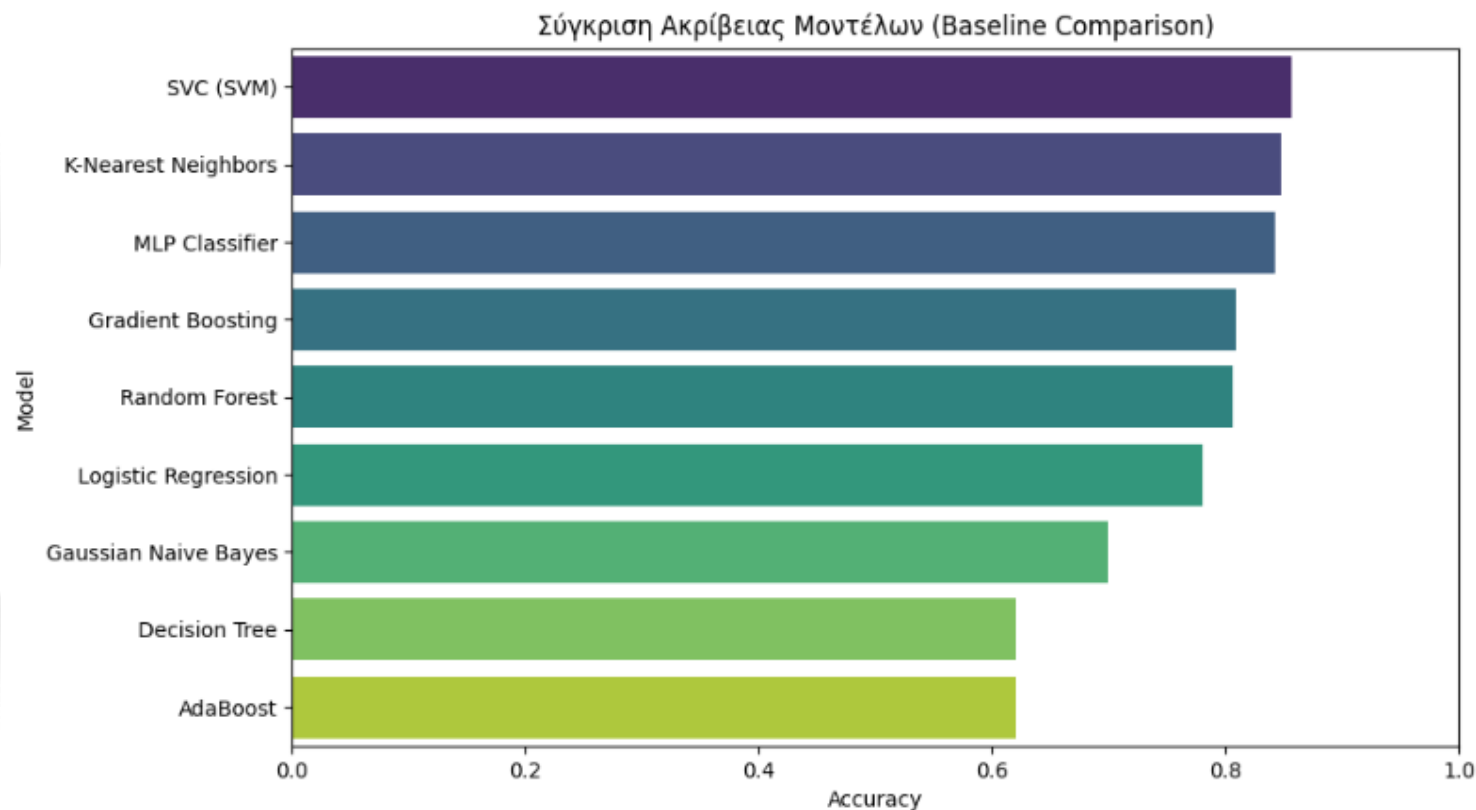
Τέλος εφαρμόζουμε τον αλγόριθμο σε νέα δεδομένα ώστε να τα διαχωρίσουμε στις 5 κλάσεις.

Ξεκινάμε την επίλυση του προβλήματος διερευνώντας το αν τα δεδομένα μας είναι ισορροπημένα. Όπως φαίνεται στο διάγραμμα έχουμε πλήρως ισορροπημένα δεδομένα. Ως συνέπεια δε θα χρειαστεί να χρησιμοποιήσουμε μεθόδους class balancing όπως SMOTE ή Tomek Links.



Επιλογή μοντέλου:

Για να επιλέξουμε ποιο μοντέλο θα χρησιμοποιήσουμε και αργότερα θα κάνουμε tune τρέξαμε το πρόβλημα από το πιο απλό στο πιο σύνθετο μοντέλο με τις default παραμέτρους του. Όπως φαίνεται παρακάτω τα καλύτερα αποτελέσματα μας τα έδωσε ο SVC. Αξιοσημείωτη είναι η επίδοση του KNN που με το default `n_neighbors=5` πετύχαμε τη μέγιστη επίδοσή του και ήρθε ελάχιστα πίσω από τον default SVC που κρύβει όμως μερικές παγίδες. Στην επόμενη διαφάνεια θα δούμε και τον χρόνο του κάθε μοντέλου. Τα αποτελέσματα είναι χωρίς scaling στα δεδομένα καθώς έριχναν την ακρίβεια, θα το αναλύσουμε παρακάτω.

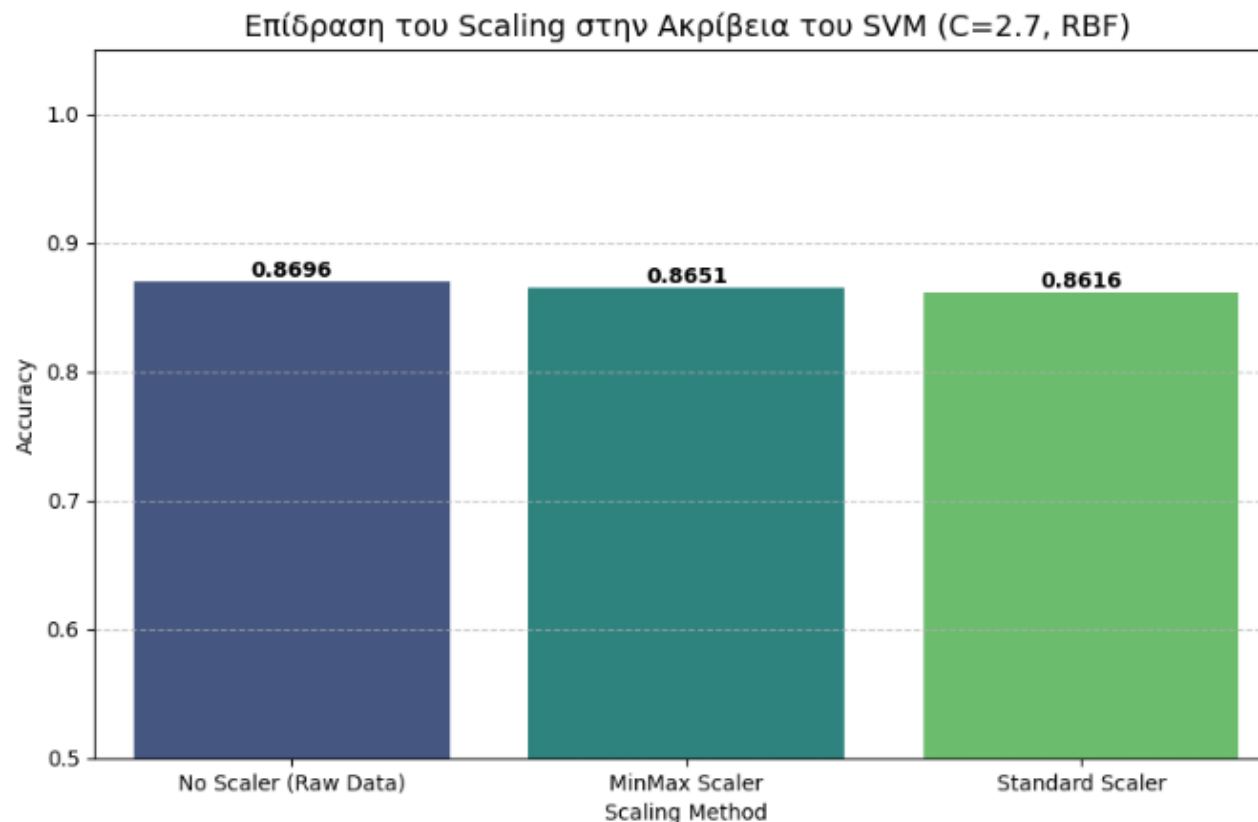


Παρατηρούμε ότι ο KNN είναι αστραπιαίος αυτό όμως είναι πλασματικό καθώς είναι lazy-learner και στην εκπαίδευση δεν κάνει τίποτα από το να αποθηκεύσει το πλέγμα των σημείων του training set. Σε real time εφαρμογές όμως δε συγκρίνεται με τον SVC ο οποίος μετά την εκπαίδευσή του είναι ταχύτατος ενώ ο KNN αργεί πολύ γιατί κάνει όλους τους υπολογισμούς του από την αρχή κάθε φορά που έρχεται ένα καινούριο sample για ταξινόμηση. Επιπλέον έχουμε φτάσει στο μέγιστο tuning γι' αυτό θα προτιμήσουμε τον SVC από τον KNN που μας δίνει και τη δυνατότητα να τον βελτιώσουμε ακόμη περισσότερο. Ακόμη ο MLPClassifier απορρίπτεται και εκείνος καθώς είναι πιο περίπλοκος και ακόμη και μετά από tuning ήταν χειρότερος από τον SVC. Αν βάζαμε πάνω από ένα hidden layer είχαμε overfitting και το accuracy έπεφτε.

Model	Accuracy	Time (sec)
SVC (SVM)	0.856489	5.315727
K-Nearest Neighbors	0.846770	0.285592
MLP Classifier	0.841624	11.681483
Gradient Boosting	0.808462	508.200938
Random Forest	0.806175	16.104655
Logistic Regression	0.779874	2.801307
Gaussian Naive Bayes	0.698685	0.042677
Decision Tree	0.620354	4.681011
AdaBoost	0.620354	18.798178

Scaling των δεδομένων

Όπως βλέπουμε στο διάγραμμα δεξιά αν κάνουμε scaling στα δεδομένα μας η ακρίβεια μειώνεται ελάχιστα. Αυτό εξηγείται από την ομοιογένεια των χαρακτηριστικών μας (pixels) και έτσι δεν υφίσταται το πρόβλημα κυριαρχίας χαρακτηριστικών με μεγάλες τιμές (όπως θα συνέβαινε σε ετερογενή δεδομένα). Συνεπώς, επιλέγουμε να χρησιμοποιήσουμε τα αρχικά δεδομένα, διατηρώντας την φυσική κλίμακα και αποφεύγοντας περιττή πολυπλοκότητα.



Παράμετροι

Μετά από tuning καταλήγουμε στον :`SVC(C=2.7, gamma='scale', kernel='rbf', random_state=42)`

Το $C=2.7$ είναι η χρυσή τομή της αυστηρότητας στο margin, δηλαδή θέλω να είμαι αυστηρός ώστε να ταξινομώ σωστά αλλά να μπορώ να γενικεύσω κιόλας. Αν το C ήταν μικρό θα δεχόμασταν αρκετά λάθη ώστε να κρατήσουμε το περιθώριο φαρδύ ενώ αν το C ήταν πολύ μεγάλο δε θα θέλαμε να κάνουμε σχεδόν κανένα λάθος στην εκπαίδευση ρισκάροντας overfitting.

Το `gamma='scale'` που δίνεται από τον τύπο $1/(n_features * X.var())$ μου προσφέρει βέλτιστη προσαρμογή με βάση την διακύμανση των χαρακτηριστικών μου. Το `gamma` γενικά δείχνει την εμβέλεια επιρροής του κάθε training sample. Αν έβαζα μεγάλο `gamma` θα είχα πολύπλοκα σύνορα και overfitting ενώ αν έβαζα μικρό `gamma` θα είχα σχεδόν γραμμικά σύνορα και underfitting. Έτσι, αν τα δεδομένα έχουν μικρή διακύμανση (είναι πυκνά), το `scale` αυξάνει αυτόματα το `gamma` για να εστιάσει στη λεπτομέρεια. Αν έχουν μεγάλη διακύμανση (είναι αραιά), μειώνει το `gamma` για να δει την ευρύτερη δομή.

Το `kernel='rbf'` επιτρέπει στον SVC να απεικονίσει τα δεδομένα σε άπειρες διαστάσεις έως ότου να καταφέρει να τραβήξει ένα υπερ-επίπεδο που να τα διαχωρίζει. Έτσι έχει τη δυνατότητα να δημιουργεί μη γραμμικά όρια απόφασης ενώ στη πραγματικότητα επιλύει πρόβλημα γραμμικού διαχωρισμού.

Αποτελέσματα στο splitted test set

Τα αποτελέσματα μας βελτιώθηκαν έως το 87% με το tuning το οποίο δεν είναι ένα άριστο ποσοστό ακρίβειας γενικά. Στην περίπτωση μας όμως αν κοιτάξουμε τον confusion matrix θα διαπιστώσουμε ότι η συντριπτική πλειοψηφία των λαθών μας είναι ανάμεσα στις κλάσεις 2 και 5, δηλαδή πολλές φορές όταν έχουμε 2 προβλέπουμε 5 και το αντίστροφο. Για την επίλυση του συγκεκριμένου προβλήματος προσπαθήσαμε διάφορες τεχνικές όπως και το stacking(ίδια αποτελέσματα με τον SVM με πολύ μεγαλύτερη πολυπλοκότητα) χωρίς όμως αποτέλεσμα καθώς όπως θα δούμε στη συνέχεια η φύση του προβλήματος είναι τέτοια που κανένα μοντέλο δε μπορεί να λύσει. Είναι το αναπόφευκτο bayes error.

Confusion Matrix

	1	2	3	4	5
1	368	3	5	0	1
2	3	245	9	10	65
3	8	6	328	7	0
4	0	11	12	292	10
5	8	63	2	5	288
	1	2	3	4	5

True label

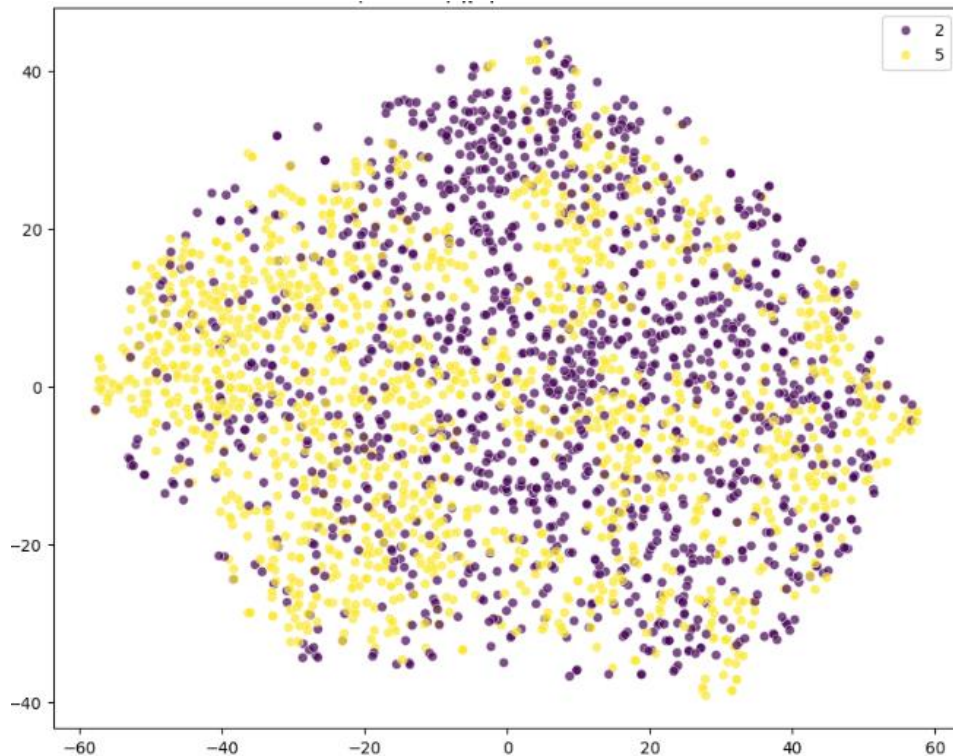
Predicted label

Accuracy: 86.96 %
Precision: 86.81 %
Recall: 86.79 %
F1 Score: 86.78 %

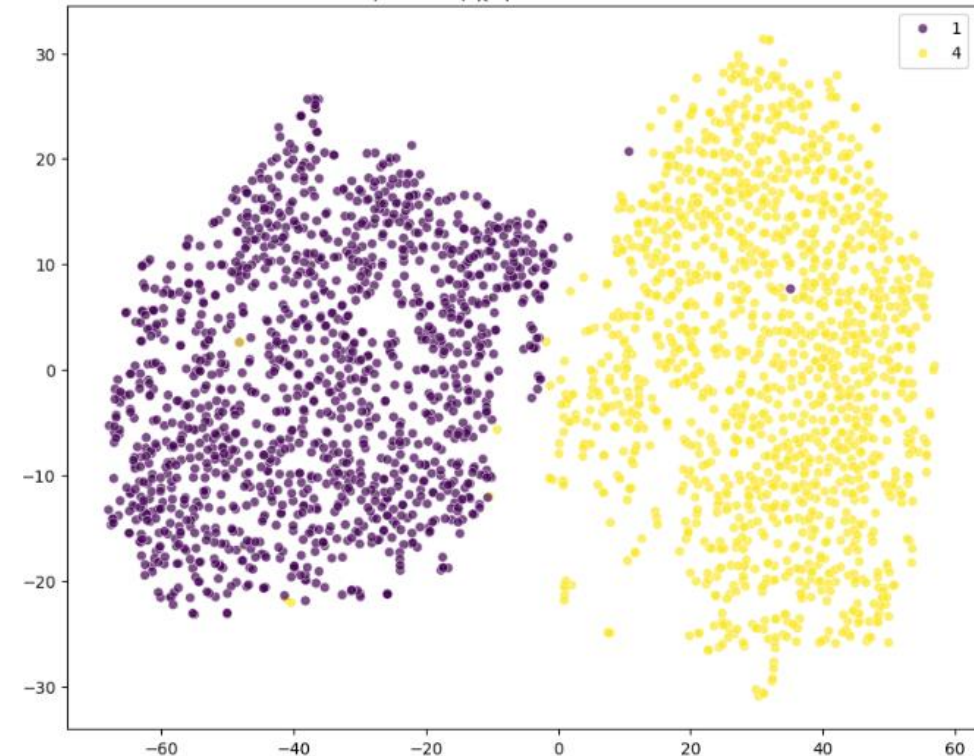
Οπτικοποίηση των κλάσεων 2,5 στις 2 διαστάσεις

Με τη χρήση του εργαλείου t-SNE που προσπαθεί όσα σημεία ήταν κοντά στις 224 διαστάσεις να τα απεικονίσει κοντά και στις 2 διαστάσεις βλέπουμε αριστερά ότι οι κλάσεις 2 και 5 είναι μια 'σούπα' με κάποιες περιοχές που υπερτερεί η μία της άλλης. Αυτό το οπτικό συμπέρασμα επιβεβαιώνεται ποσοτικά στη συνέχεια με την ανάλυση του *Cohen's d*. Στην ιδιότητα αυτή οφείλεται και το σφάλμα ταξινόμησης που έχουμε. Ενώ αν συγκρίνουμε με τις κλάσεις 1 και 4 βλέπουμε πως διαχωρίζονται τέλεια καθώς οι περιοχές τους είναι εντελώς διαφορετικές.

Οπτικοποίηση κλάσεων 2, 5

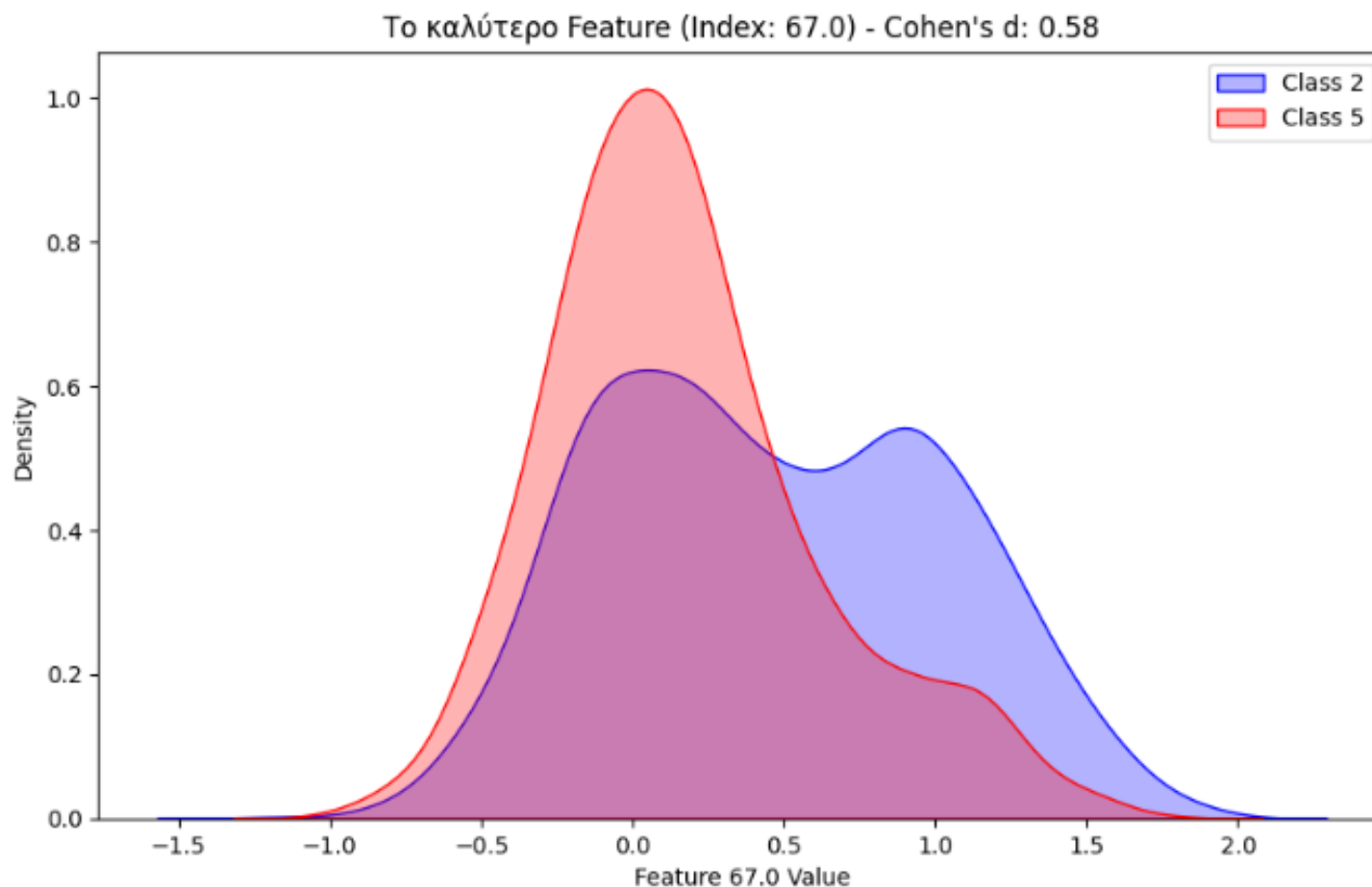


Οπτικοποίηση κλάσεων 1, 4



Cohens d

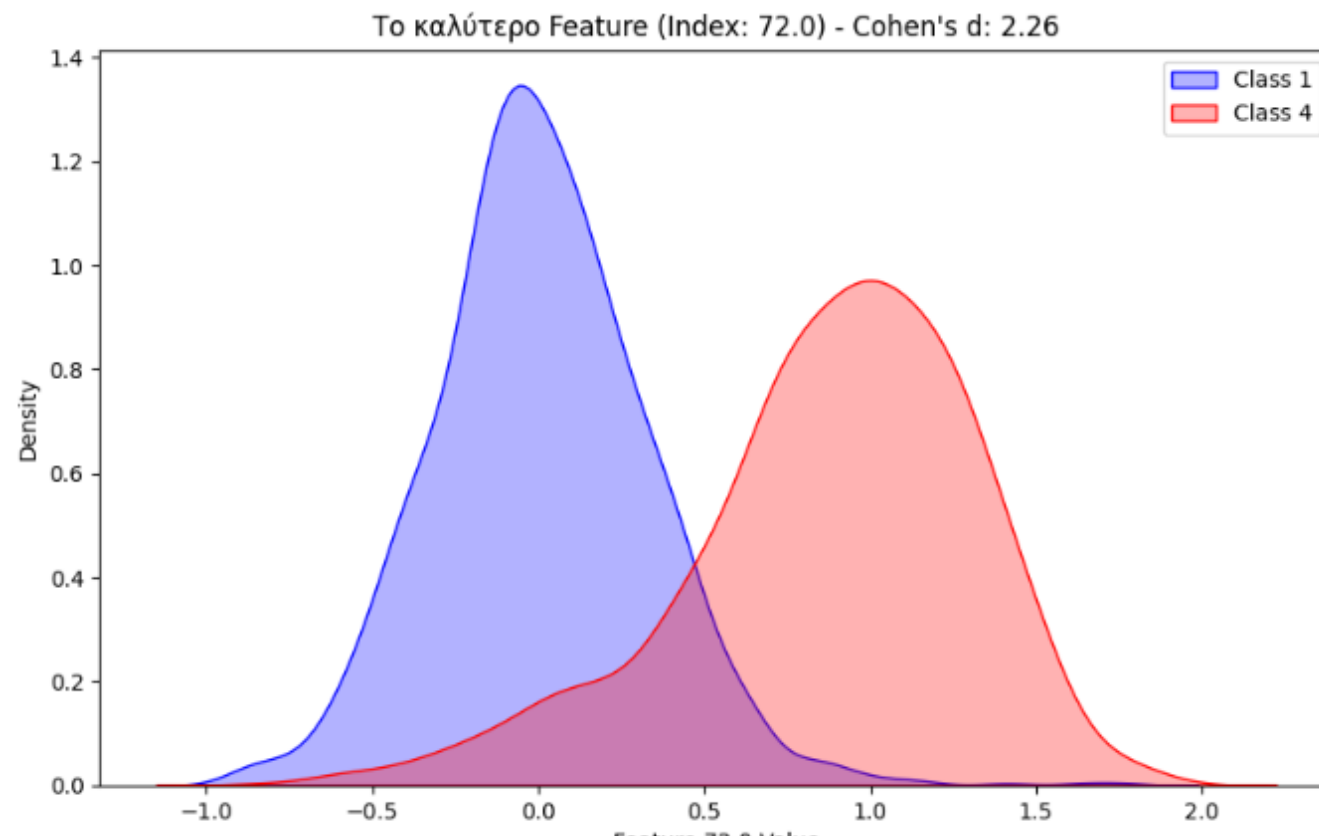
Άλλο ένα μέγεθος που μας εξηγεί το μπέρδεμα στη ταξινόμηση των κλάσεων 2 και 5 είναι το Cohens d που είναι η διαφορά του μέσου όρου ενός συγκεκριμένου feature δεδομένου ότι ανήκει στη κλάση 2 με το ίδιο feature δεδομένου ότι ανήκει στη κλάση 5 διαιρεμένη με το pooled std. Το μέγεθος αφορά κάθε feature ξεχωριστά. Αν έχουμε features που έχουν cohens d > 0.8 έχουμε ελπίδα για μία καλή ταξινόμηση ενώ αν τα καλύτερα feature όπως και στη περίπτωση μας έχουν cohens d < 0.6 έχουμε μία πιο μέτρια ταξινόμηση. Δεξιά βλέπουμε τη πιθανοφάνεια του πιο δυνατού feature για το διαχωρισμό των 2 κλάσεων $p(x_{67}|w_2)$ και τη $p(x_{67}|w_5)$. Παρατηρούμε ότι η μπλε συνάρτηση επικαλύπτεται σε πολύ μεγάλο ποσοστό από τη κόκκινη και γι αυτό το λόγο υπάρχει και το σφάλμα.



```
--- TA 5 ΚΑΛΥΤΕΡΑ FEATURES ΓΙΑ ΔΙΑΧΩΡΙΣΜΟ 2 VS 5 ---
feature_index  cohens_d  mean_diff
67             67      0.577664  0.295414
100            100      0.548614  0.298923
21             21      0.541983  0.292094
175            175      0.483129  0.261592
147            147      0.473596  0.261581
```

Σύγκριση ξανά με cohens d για 1, 4

Είναι φανερό η διαφορά, εδώ βλέπουμε ότι τα top5 features έχουν cohens d >1.9. Οι κατανομές πιθανοφάνειας για το καλύτερο feature έχουν ξεκάθαρες κορυφές που δεν επικαλύπτονται. Γι αυτό έχουμε και πολύ καλύτερη ταξινόμηση

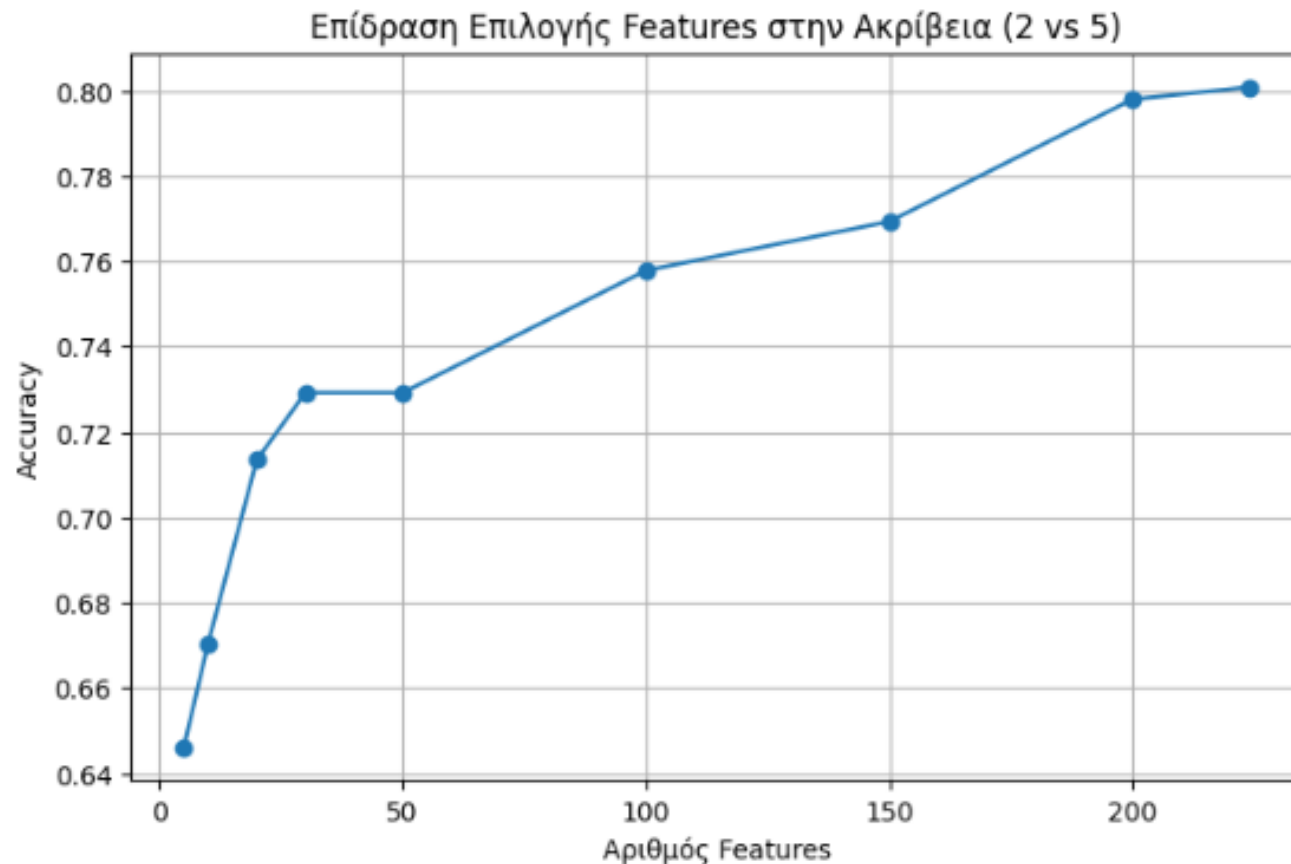


--- TA 5 ΚΑΛΥΤΕΡΑ FEATURES ΓΙΑ ΔΙΑΧΩΡΙΣΜΟ 1 VS 4 ---

	feature_index	cohens_d	mean_diff
72	72	2.261356	0.865751
222	222	2.254583	0.881164
173	173	2.180930	0.861334
189	189	2.123183	0.859386
43	43	1.926505	0.802535

Επίδραση επιλογής features στην ακρίβεια για κλάσεις 2 vs 5

Μια αρχική σκέψη ήταν η υλοποίηση ενός μοντέλου Ιεραρχικής Ταξινόμησης (το πρώτο μοντέλο κάνει ένα αρχικό διαχωρισμό σε 4 κλάσεις και το δεύτερο κάνει ένα binary). Στόχος ήταν η απομόνωση των δύσκολων κλάσεων (2 και 5) και η εκπαίδευση ενός εξειδικευμένου binary classifier, ο οποίος θα χρησιμοποιούσε μόνο ένα υποσύνολο ουσιαστικών χαρακτηριστικών (μεγαλύτερα cohen's d) για να μειώσει τον θόρυβο. Σαν δεύτερα μοντέλα δοκίμασα αρκετά ώστε να καταφέρουν να πιάσουν διαφορετικές σχέσεις στα δεδομένα. Το εγχείρημα αυτό όμως δε δούλεψε λόγω του διπλανού διαγράμματος που δείχνει ότι χρειαζόμαστε όλα τα features ώστε να πετύχουμε τη μέγιστη ακρίβεια και λόγω του αναπόφευκτου bayes error που σχολιάσαμε στις προηγούμενες διαφάνειες και είτε χρησιμοποιήσουμε εξειδικευμένο ταξινομητή είτε όχι μόνο πολυπλοκότητα προσθέτουμε δίχως αύξηση της ακρίβειας.



Στο παραπάνω διάγραμμα έχουμε χρησιμοποιήσει τον tuned SVC που έχουμε αναφέρει αποκλειστικά για τα δείγματα που αφορούν τις κλάσεις 2 και 5. Στο παρακάτω διάγραμμα φαίνονται οι ακριβείς τιμές. Παίρνουμε κάθε φορά τα feature με το **μεγαλύτερο** cohen's d.

```
Top 5 Features (Lowest d = 0.474) -> Accuracy: 0.6461
Top 10 Features (Lowest d = 0.432) -> Accuracy: 0.6705
Top 20 Features (Lowest d = 0.365) -> Accuracy: 0.7135
Top 30 Features (Lowest d = 0.305) -> Accuracy: 0.7292
Top 50 Features (Lowest d = 0.234) -> Accuracy: 0.7292
Top 100 Features (Lowest d = 0.115) -> Accuracy: 0.7579
Top 150 Features (Lowest d = 0.043) -> Accuracy: 0.7693
Top 200 Features (Lowest d = 0.015) -> Accuracy: 0.7980
Top 224 Features (Lowest d = 0.000) -> Accuracy: 0.8009
```